

ENTREGA INICIAL

Ambiente, Especificação Técnica e Test Harness

Projeto: SGC - Sistema de Gerenciamento de Chamados

Disciplina: [NOME DA DISCIPLINA]

Professor: [PROFESSOR]

Integrante 1: [NOME DO ALUNO 1] - RA [RA DO ALUNO 1]

Integrante 2: [NOME DO ALUNO 2] - RA [RA DO ALUNO 2]

Repositório: [LINK DO GITHUB]

Documento pré-preenchido. Antes da submissão, substituir os placeholders e inserir evidências reais do GitHub, Docker, agente e CI.

1. Resumo do problema

Pequenas equipes de suporte frequentemente controlam solicitações por mensagens ou planilhas. Isso reduz rastreabilidade, dificulta priorização, atribuição de responsável e acompanhamento de estados. O SGC propõe uma API REST pequena, reproduzível e testável para registrar e acompanhar chamados.

2. Solução e escopo

A API permite criar, listar e consultar chamados, alterar status segundo transições válidas, alterar prioridade e atribuir responsável. Autenticação, notificações e dashboard foram deliberadamente mantidos fora do escopo desta entrega inicial para evitar complexidade sem relação direta com os critérios avaliados.

3. Stack e ambiente

Python 3.12, FastAPI, Pydantic, SQLAlchemy, SQLite, pytest, Docker, Docker Compose e GitHub Actions. O ambiente foi definido para permitir execução local ou via containers com dependências declaradas em requirements.txt.

4. Arquitetura

Arquitetura em camadas: Routes -> Services -> Repositories -> SQLAlchemy -> SQLite. Schemas Pydantic validam contratos de entrada e saída. Regras de negócio permanecem no Service, persistência no Repository e comunicação HTTP nas Routes.

5. Fluxo SDD

A especificação técnica é mantida em docs/specification/specification.md. A implementação parte de requisitos identificados, define contratos e regras, cria/ajusta testes, implementa a menor alteração coerente, executa o harness e somente então segue para Pull Request e revisão.

6. Agente de IA

Agente de referência: Codex CLI. O repositório contém instruções versionadas em .ai/instructions e prompts em .ai/prompts. As regras exigem leitura da especificação, preservação da arquitetura, criação de testes e proíbem declarar execuções inexistentes.

7. Test Harness

O harness usa pytest, FastAPI TestClient e banco SQLite em memória isolado por teste. Comando local: `pytest --junitxml=docs/test-results/junit.xml`. Comando padronizado: `docker compose run --rm tests`.

8. Execução real realizada neste ambiente

A suíte foi executada localmente em 08/09/2026. Resultado real: 16 testes aprovados, sem falhas. Os arquivos docs/test-results/pytest.log e docs/test-results/junit.xml foram gerados pela execução. Docker não estava disponível neste ambiente, portanto nenhuma execução Docker foi simulada.

9. Requisitos principais

ID	Descrição
RF01	Criar chamado válido com status inicial ABERTO
RF02	Listar chamados
RF03	Consultar chamado por ID
RF04	Alterar status respeitando transições
RF05	Alterar prioridade
RF06	Atribuir responsável

ID	Descrição
RF07	Validar entradas inválidas

10. Matriz resumida de testes

Cenário	Rastreabilidade	Esperado
Criação válida	RF01 / RN01	201 / ABERTO
Lista vazia/com dados	RF02	200
Consulta inexistente	RF03	404
Fluxo de status	RF04 / RN02-RN04	200
Transição inválida	RN02-RN04	409
Entradas inválidas	RF07 / RN07	422
Alterar encerrado	RN05	409
Health check	RNF05	200

11. Governança GitHub

Fluxo definido: main (estável), develop (integração) e feature/* (tarefas independentes). O processo esperado é Issue -> feature branch -> Pull Request -> revisão/aprovação de outro integrante -> merge em develop -> PR para main. O pacote contém templates para Issues, Pull Request e workflow de CI.

AÇÃO NECESSÁRIA PELO ALUNO: criar o repositório público, branches, Project/Issues, realizar commits e Pull Requests reais, obter revisão/aprovação e capturar as evidências.

12. Evidências obrigatórias antes do Moodle

1. Repositório público e árvore do projeto
2. Branches main/develop/feature/*
3. GitHub Project/Issues com divisão de tarefas
4. Pull Request com comentários e aprovação
5. Uso/configuração real do agente de IA
6. Docker em execução
7. Test Harness executado no ambiente padronizado
8. Relatório/log do harness
9. GitHub Actions com pipeline concluído

13. Matriz de conformidade

Requisito	Situação
README / SDD / ADRs / contratos	CRIADO
Código e Test Harness	CRIADO E EXECUTADO LOCALMENTE
16 testes automatizados	APROVADOS LOCALMENTE
Dockerfile / Compose	CRIADO; EXECUÇÃO PENDENTE EM MÁQUINA COM DOCKER
GitHub Actions	CRIADO; EXECUÇÃO REAL PENDENTE
Branches / Issues / PRs	DOCUMENTADOS; CRIAÇÃO REAL PENDENTE
Evidências e identificação	PENDENTE DO ALUNO

14. Conclusão

O pacote fornece aplicação funcional, especificação SDD, decomposição em camadas, regras de agente de IA, ambiente Docker, CI, templates de governança e harness automatizado. A suíte foi validada localmente com 16 testes aprovados. A submissão final somente deve ser marcada como completa após a equipe produzir as evidências reais de GitHub, Docker, agente e pipeline e preencher os dados acadêmicos.

IMPORTANTE: este PDF é uma versão pré-preenchida. Não submeter sem substituir placeholders e inserir prints/logs reais exigidos pelo enunciado.